arXiv:2602.02385v1 [cs.LG] 2 Feb 2026

Transformers learn factored representations

Adam Shai [1] **Loren Amdahl-Culleton** [1] **Casper L. Christensen** [1] **Henry R. Bigelow** [1] **Fernando E. Rosas** [2 3 4]
Alexander B. Boyd [5] **Eric A. Alt** [1] **Kyle J. Ray** [1] **Paul M. Riechers** [1]

Abstract

Transformers pretrained via next token prediction learn to factor their world into parts, representing these factors in orthogonal subspaces of the residual stream. We formalize two representational hypotheses: (1) a representation in the product space of all factors, whose dimension grows exponentially with the number of parts, or (2) a factored representation in orthogonal subspaces, whose dimension grows linearly. The factored representation is lossless when factors are conditionally independent, but sacrifices predictive fidelity otherwise, creating a tradeoff between dimensional efficiency and accuracy. We derive precise predictions about the geometric structure of activations for each, including the number of subspaces, their dimensionality, and the arrangement of context embeddings within them. We test between these hypotheses on transformers trained on synthetic processes with known latent structure. Models learn factored representations when factors are conditionally independent, and continue to favor them early in training even when noise or hidden dependencies undermine conditional independence, reflecting an inductive bias toward factoring at the cost of fidelity. This provides a principled explanation for why transformers decompose the world into parts, and suggests that interpretable low dimensional structure may persist even in models trained on complex data.

[1]Simplex, Astera Institute [2]Sussex AI and Sussex Centre for Consciousness Science, University of Sussex [3]Centre for Complexity Science and Center for Psychedelic Research, Department of Brain Sciences, Imperial College London [4]Center for Eudaimonia and Human Flourishing, University of Oxford [5]Beyond Institute for Theoretical Science (BITS). Correspondence to: Adam Shai <adamimos@gmail.com>, Paul Riechers <pmriechers@gmail.com>.

Preprint. February 3, 2026.

1. Introduction

It's easy to take for granted that our world is not monolithic, but is made of parts. But for ourselves and AI models like transformers, we merely perceive the world via a stream of undifferentiated inputs. Humans take this stream of raw data, and decompose it into discrete objects—chairs, coffee mugs, other agents—in order to understand the world. Do transformers also learn to factor their world into parts? How can they from a mere stream of tokens, and why would they?

In this paper, we show that neural networks like transformers indeed learn and have an inductive bias for factored representations as a natural consequence of next-token pretraining. We rigorously test the nature of factorization in transformer representations by presenting a formal framework connecting training data structure to factored representations in neural networks. We use this framework to generate competing geometric hypotheses, and our experiments test which geometry transformers learn. Overall, this work brings three main contributions:

1. **A theoretical framework linking latent factorization in the data-generating process to activation geometry**. This provides a rigorous bridge between compositional structure in the data-generating process and properties of the activations of neural networks, including the existence of orthogonal subspaces, their dimensionality, and the geometry of context embeddings within them.

2. **Empirical confirmation that transformers learn factored structure.** When the data-generating process admits a decomposition into conditionally independent factors, transformers learn factored representations in orthogonal subspaces, achieving the predicted exponential dimension reduction even when model capacity would easily accommodate the full unfactored representation.

3. **Evidence for an inductive bias toward factored representations.** When the data-generating process is not factorizable, transformers nevertheless first learn a lossy factored representation before gradually expanding dimensionality to recover fidelity. The factored solution acts as a representational attractor: models

find it quickly, dwell there, and depart only under sustained pressure from the loss gradient. This reveals that transformers actively seek to factorize representations more generally.

Figure 1 provides an overview of the framework and main results.

2. From training data to activation geometry

In order to test if transformers have representations that correspond to parts in some way, we begin by establishing a theoretical framework which mathematically captures the relationship between the sequences of tokens that make up training data, and the expected internal representations in networks trained to do next token predictions on that data. First, we establish what constitutes *latent structure* in training data, and from there define what that structure looks like in a factored and non-factored form. Our testable predictions then come from establishing the geometry implied by the task of next-token prediction on those two generators.

2.1. Latent data-generators

What constitutes the structure of training-data in the context of standard next-token prediction, and how might it constrain what a network learns? Consider length-L token sequences $x_{1:L} = (x_1, \ldots, x_L) \in \mathcal{X}^L$, sampled from a ground-truth joint distribution $Q(X_{1:L})$. These sequences will generally be non-Markovian, such that predicting the next-token optimally may require information in arbitrarily long contexts.

We conceptualize these sequences as being generated by a latent process that evolves over time and emits observed tokens. To make this precise, we use generalized Hidden Markov Models (GHMMs). Any distribution over sequences admits a GHMM representation, so this is fully general. We will use this to derive exact predictions about transformer representations in the following subsections. A GHMM is defined by the tuple:

$$\mathcal{M} = \left(\mathcal{X}, \boldsymbol{\mathcal{S}}, \boldsymbol{\eta}^{(\varnothing)}, (T^{(x)})_{x \in \mathcal{X}}\right) \tag{1}$$

where $\mathcal{X}$ is the token alphabet, $\boldsymbol{\mathcal{S}} = \mathbb{R}^d$ is the latent space, $\boldsymbol{\eta}^{(\varnothing)}$ is an initial state vector in $\mathbb{R}^d$, and $T^{(x)}$ is the operator describing latent dynamics for each token x. For GHMMs, the probability of any sequence can be calculated as

$$Q_{\mathcal{M}}(x_{1:\ell}) = \boldsymbol{\eta}^{(\varnothing)} T^{(x_1)} \cdots T^{(x_\ell)} \mathbf{1} \ , \tag{2}$$

where $\mathbf{1}$ is the vector that integrates probability in the latent space. GHMMs include hidden Markov models (HMMs) as a special case[1], but also include more general generators. For details, see Appendix A.1.

2.2. Geometry associated with latent generators

After observing a context $x_{1:L}$, what does the model need to know to predict optimally? Only the information the past reveals about the future (Upper, 1997). We call the vector encoding this information the *predictive vector*:

$$\boldsymbol{\eta}^{(x_{1:\ell})} := \frac{\boldsymbol{\eta}^{(\varnothing)} T^{(x_1)} \cdots T^{(x_\ell)}}{\boldsymbol{\eta}^{(\varnothing)} T^{(x_1)} \cdots T^{(x_\ell)} \mathbf{1}} \ . \tag{3}$$

Two contexts with identical predictive vectors make identical predictions about all future tokens. Contexts with *similar* predictive vectors make *similar* predictions, and prior work has shown this similarity structure is reflected geometrically in transformer activations (Shai et al., 2024).

The collection of predictive vectors across all possible contexts forms a geometric arrangement in the latent space, which is determined by the structure of the data-generating process (Figure 1a, bottom). For a d-dimensional latent space, this arrangement takes place in $d-1$ dimensions (since predictive vectors are normalized). For more discussion and examples, see Appendices A.2 and B.

2.3. Conditionally independent data-generators

Consider a data-generating process that has *parts*, composed of multiple processes running in parallel. An example is shown in Figure 1a. Each part has its own latent space, and each contributes to the observed output. **Crucially, these parts are not visible at the token level.**[2] The factorization into parts exists purely in the latent structure of the generator. Discovering it requires inferring, from the correlational structure of token sequences alone, that the world decomposes into parts.

We call such generators *conditionally independent*: given the observed token, each factor's latent dynamics evolve independently. The factors may influence each other's outputs, but once you condition on what was actually emitted, they decouple. This can be formalized in terms of GHMMs:

Definition 2.1. *A GHMM generator is* ***conditionally independent*** *if each token-labeled linear-operator can be expressed as the tensor product of multiple linear operator factors:*

$$T^{(x)} = \bigotimes_{n=1}^{N} T_n^{(x)} \ . \tag{4}$$

Such a multi-partite generator has latent space $\mathcal{S} = \bigotimes_{n=1}^{N} \mathcal{S}_n$, the tensor product of N constituent latent spaces.

[1] In this case, there are d hidden states, and substochastic transition matrices $T^{(x)} : \mathbb{R}^d \to \mathbb{R}^d$ whose matrix elements are simply token-dependent transition probabilities between latent states. This provides helpful intuition for many (but not all) purposes.

[2] Through training, the model sees only the token, and nothing signals that, e.g., 'token 347' is secretly 'sub-token 2 from process 1, sub-token 0 from process 2, etc'.

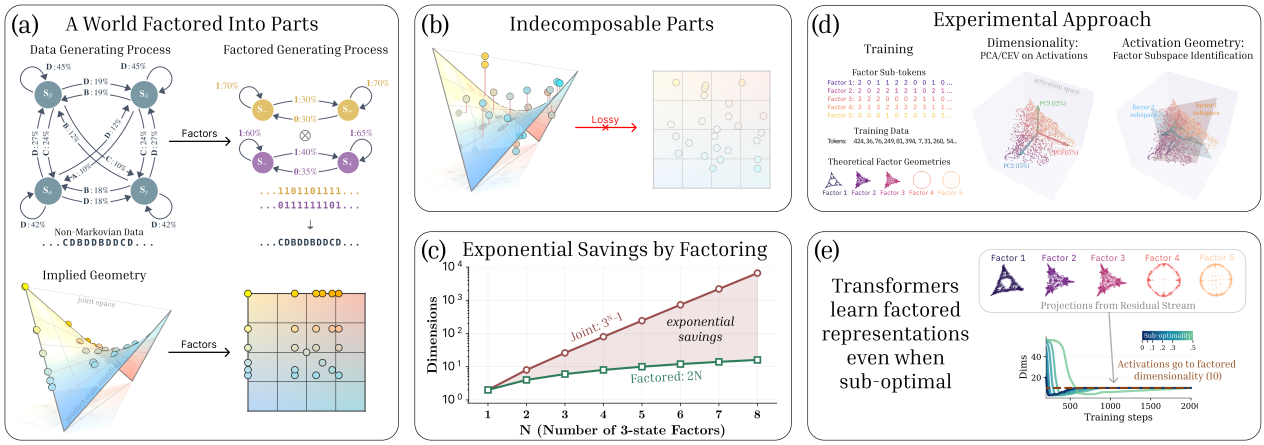

Figure 1. Transformers learn to factor the world into parts, representing each factor in orthogonal subspaces. **(a)** Consider a data-generating process with 4 hidden states (top-left). Although the joint process appears complex, it may admit a representation as a product of simpler independent processes running in parallel (top-right). Below, we illustrate how this factorization affects the geometry of representations: the full simplex over joint latent states (a tetrahedron for four joint states) decomposes into the product of simplices for each factor (two line segments). Even in cases where parts influence each other's dynamics, as long as conditioning on observed tokens renders the factors independent, the same geometric decomposition applies: uncertainty over each factor can be tracked separately in orthogonal subspaces. **(b)** When factors remain correlated even after conditioning on observations, the joint state cannot be written as a product state—it lies off the product-state manifold. Projecting onto the factored representation (red arrow) is lossy: some predictive information is sacrificed for the dimensional savings. **(c)** This decomposition yields dramatic representational savings. For N three-state factors, the joint representation requires $3^N - 1$ dimensions (red), while the factored representation requires only $2N$ dimensions (green)—an exponential reduction. **(d)** To determine if transformers learn factored representations, we construct training data that is latently composed of sub-tokens generated from 5 factors. Each of these factors has a geometric prediction from our theory. Our analysis methods consist of quantifying the dimensionality of the activations using PCA and explained variance, as well as determining the geometric arrangement of factor subspaces. **(e)** Our main results are that transformers learn factored representations, placing these representations in orthogonal subspaces of the residual stream. Crucially, transformers do this even when it hurts prediction.

The dimension of this joint latent space grows exponentially with the number of components. N factors with m-dimensional latent space yield a joint space of dimension m^N.

Conditionally independent generators realize a wide variety of models of interest. In our experiments, we consider N processes that emit unobserved sub-tokens $x^{(1)}, \ldots, x^{(N)}$, each with their own generator. The observed vocabulary $\mathcal{X}$ has a unique token for each collection of sub-tokens; nothing in the training data explicitly signals the underlying factored structure.

2.4. Joint vs. factored representations

We now arrive at our central question: what representational geometry should we expect to find in a network trained on data from a world with parts? Two possibilities are mathematically natural:

The joint representation tracks predictive vectors in the full tensor-product space (Figure 1a, bottom left). For N factors with latent dimensions $d_1, \ldots, d_N$, this requires $(\prod_n d_n) - 1$ dimensions, but is always lossless.

The factored representation tracks predictive vectors over each factor separately, in orthogonal subspaces (Figure 1a, bottom right). This requires only $\sum_n (d_n - 1)$ dimensions—exponentially fewer (Figure 1c), but sacrifices fidelity when factors aren't conditionally independent (Figure 1b).

For the task of predicting future tokens, the factored representation is sufficient when a key property holds: when the generator is conditionally independent, latent states that factorize across parts remain factored through context. More precisely:

Proposition 2.2. *When the transition operators are conditionally independent, the predictive vector remains a* product state*:*

$$\boldsymbol{\eta}^{(x_{1:\ell})} = \bigotimes_{n=1}^{N} \boldsymbol{\eta}_n^{(x_{1:\ell})} \implies \boldsymbol{\eta}^{(x_{1:\ell+1})} = \bigotimes_{n=1}^{N} \boldsymbol{\eta}_n^{(x_{1:\ell+1})}. \quad (5)$$

Here, *product state* refers to a special class of predictive vectors in $\mathbb{R}^{\prod_n d_n}$ that can be expressed as a tensor product over the state of each factor. These product states form a low-dimensional sub-manifold within the full joint space (bottom left of Figure 1a), and conditionally independent dynamics keep beliefs on this sub-manifold.[3] Importantly, this implies that, when the token-induced transition dynamic

[3]More precisely, the product-state sub-manifold maps to itself under conditionally independent generators, and the projective